

The Human–VLA Communication Gap: What Humans Omit and Policies Cannot Recover

Anonymous Author(s)

Affiliation withheld for double-anonymous review

Abstract—A robot may possess the physical skills required for a task yet fail when a human asks for it naturally. For generalist VLAs to be accessible to non-expert users, they must handle the open-ended and often underspecified ways people express physical objectives, rather than requiring users to provide execution-ready commands. We study this human–VLA communication gap by separating an *information gap*, where executable task structure must be recovered, from a residual *execution gap*, where fully specified information may still fall outside the policy’s learned execution interface. We characterize how people naturally specify demonstrated objectives and evaluate the same physical tasks under controlled specification variants. As tasks become more complex, people increasingly compress their instructions, relying more on contextual grounding and leaving execution structure implicit. The VLA is comparatively tolerant of grounding dependence, but substantially less reliable when task semantics or event structure must be reconstructed, and these costs grow with task complexity. Even after the information gap is resolved, behavior remains sensitive to how the resolved task is organized and from what state it is invoked. By localizing where communication fails, our analysis identifies concrete targets for future policies and translation interfaces: recovering the semantics and execution structure humans naturally leave implicit, then presenting the resolved task through an interface the policy can reliably execute.

Index Terms—Vision-language-action models, human intent specification, human–robot communication.

I. INTRODUCTION

Imagine asking a household robot, “Can you clear the table so I can work here?” The robot has already learned how to pick up cups, move plates, and place objects into containers, yet it freezes. Replace the request with “Pick up the white mug and place it in the bin,” and the same policy succeeds. What failed was not the physical skill but the interface to it: the human expressed an objective naturally, while the policy expected a narrow specification of what to do, on which object, and toward what target state.

This gap grows more important as vision-language-action (VLA) models become more general-purpose. Recent VLAs acquire diverse manipulation behaviors from large collections of robot demonstrations and are evaluated by the breadth of tasks, objects, and environments they handle. Existing work has therefore focused on the *robot side* of generality: how many skills are acquired, how well they compose, and how robustly they transfer. Yet broad physical competence alone does not make a generalist policy accessible to ordinary users. A non-expert should not need to know the command templates

used during robot training. Humans communicate physical objectives through direct actions, desired outcomes, contextual references, or indirect requests. A useful generalist robot must therefore bridge not only a physical generalization gap but a communication gap between how people express intent and how the policy expects that intent to be specified.

We study this human–VLA communication gap through two successive mismatches. A natural specification leaves some of the executable task structure unstated, which the policy must recover: an *information gap*. And even a complete specification may fail if its resolved events are organized or invoked outside the policy’s learned execution interface: an *execution gap*. Quantifying them serves two purposes. First, it gives a human-centered measure of generalist capability: rather than asking only whether a policy can execute a task under a benchmark instruction, we ask how reliably the same capability can be reached by a non-expert expressing the task naturally. Second, identifying *which* executable structure is left unresolved under different forms of human specification, and what each costs the policy, gives a diagnostic account of instruction-following failure that a success rate alone cannot.

Studying this gap is difficult for three reasons. First, natural human task specification is open-ended: the same objective can be communicated through direct commands, desired outcomes, or partial references, and no fixed list of paraphrases captures it. Rather than enumerate, we begin from how people actually instruct a robot. We collect natural instructions from participants shown task demonstrations and, informed by these and by prior work on situated instruction, develop a taxonomy that factors any specification by what task content it states, how it refers to objects, and how it composes multiple manipulations. The taxonomy does two things. It makes the space generable, so that we can produce human-like instructions for any demonstrated task that vary what is made explicit while holding the physical objective fixed. And it makes the space measurable, so that we can ask what people naturally state, compress, or leave implicit as tasks grow more complex.

Second, resolving a human specification requires a policy-facing reference for what a fully explicit command looks like. Although VLAs inherit large vision-language backbones, their robot-training supervision is dominated by concise, action-centric instructions with regular structure in actions, object references, and destination descriptions [1]. We therefore mine recurring forms from robot-training datasets and use them to render each grounded event as an explicit action–object–destination command. These policy-facing realizations provide

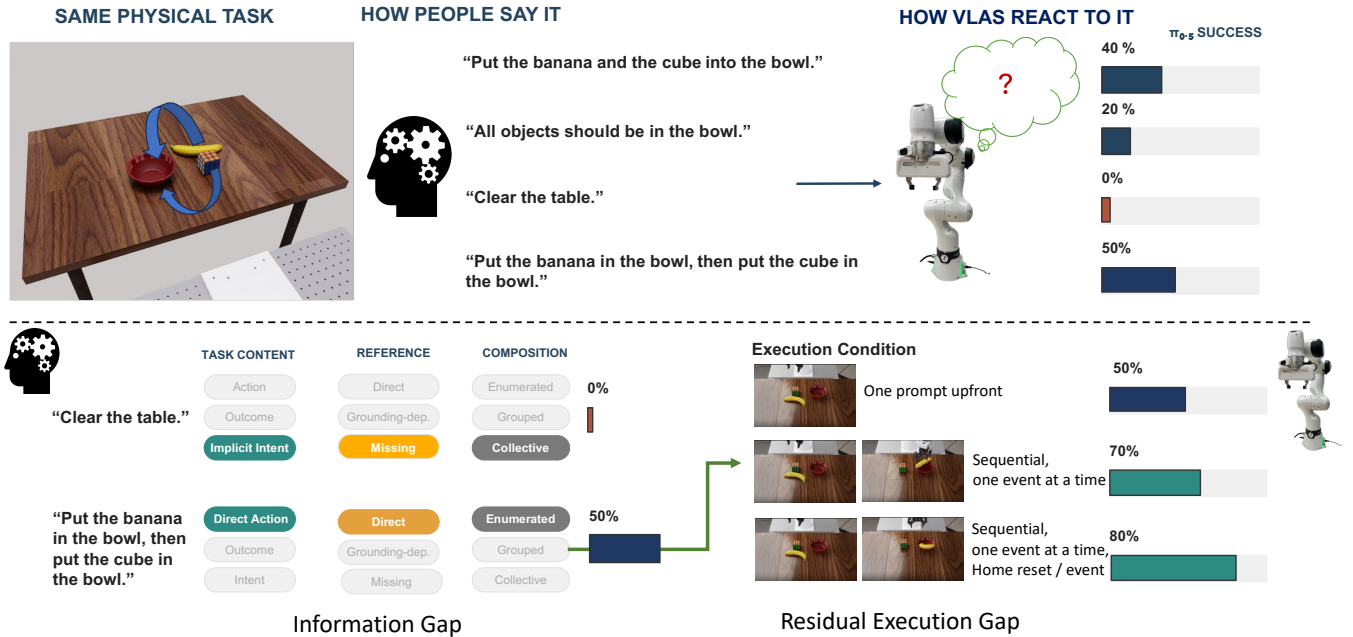


Fig. 1: Human-VLA communication gap. For the same physical task, people may express the objective with different task content, references, and levels of composition, leading to substantially different VLA performance (top). We decompose this mismatch into two stages (bottom). The *information gap* captures executable task structure that is missing, grounding-dependent, or compressed in the human specification. After this structure is fully resolved, a *residual execution gap* remains: the same task is executed more reliably when commands are issued in a policy-facing form and from a familiar invocation state. Together, these gaps separate recovering *what* the human intends from presenting it in the form under which the VLA learned to act.

a common reference against which human specifications can be compared, and allow compound objectives to be expressed as sequences of atomic events when we study execution after the information gap has been removed.

Third, a success rate only says that a policy failed, but does not locate the gap. We introduce an *information-gap analysis* that characterizes, for each specification, which executable variables are explicitly provided, which require further grounding, and whether compound manipulations are decomposed into individual events. We compare each specification against a fully grounded command that explicitly identifies every action, target object, and target placement, and execute the same physical tasks under each form, measuring task success and partial progress. This allows us to relate the type and amount of unresolved structure to the VLA’s ability to recover the intended behavior. Once this information is fully resolved, we then probe the residual *execution gap* by holding the grounded event sequence fixed while varying whether events are issued together or sequentially, and whether each event begins from the preceding or home robot state. This separates task recovery from execution once the task is fully specified.

Together, our study separates *what a VLA can physically do* from *what a human can reliably ask it to do*, and finds the two mismatched in a specific way. As tasks grow, people increasingly rely on contextual grounding and compress multiple manipulations into higher-level expressions, leaving more executable structure implicit. On the other hand, the policy handles these omissions selectively. Information that is present but requires grounding is comparatively well tolerated, whereas genuinely missing task semantics and implicit execu-

tion structure incur much larger costs, especially for complex tasks. Nor does resolving this information gap guarantee reliable execution. Even fully specified atomic events are executed more reliably when delivered one at a time and invoked from a familiar state. More broadly, these gaps identify concrete targets for future policies and translation interfaces: recovering the semantics and execution structure humans naturally leave implicit, then presenting the resolved task at a granularity and from a state in which the policy can reliably act.

II. RELATED WORK

A. Human Objective Specification and Situated Instruction.

Natural task-oriented communication extends beyond direct imperative commands. Work on speech acts distinguishes literal utterances from intended communicative acts, explaining why requests may be expressed indirectly [2], [3]. Instruction-following studies further show that speakers verbalize different parts of a task: purpose clauses convey why an action should be performed [4], preventative expressions specify states or behaviors to avoid [5], and situated references rely on shared perceptual context to identify entities [6]. Human-robot dialogue corpora likewise contain mixtures of action semantics, spatial reference, and interaction context [7]–[9]. We build on these observations by treating them as alternative strategies for specifying a common *physical objective*. Rather than studying indirectness, reference, or abstraction separately, our taxonomy characterizes what executable task information a specification exposes, how its referents are grounded, and how multiple manipulations are organized. This lets us measure both how people naturally compress physical objectives and how current robot policies respond to the resulting implicit structure.

B. The Learned Language Interface of VLA Policies.

The language paired with robot demonstrations is substantially narrower than unrestricted human communication. Audits of RT-1 [10], BridgeData V2 [11], TacoPlay [12], Language Table [13], LIBERO [14], and related datasets find highly repetitive, dataset-specific instruction distributions [1]. This contrasts with language-focused embodied datasets such as ALFRED [15] and SCOUT [9], which contain richer task descriptions or dialogue. Correspondingly, VLA behavior can be highly sensitive to linguistic realization. LIBERO-Para finds substantial performance degradation under meaning-preserving changes to action and object expressions [16]; LangGap shows that strong task performance can coexist with weak behavioral dependence on language [17]; and other studies identify gaps between semantic understanding and reliable behavioral execution under linguistic variation [18], [19]. These works largely probe whether an already explicit task survives changes in wording or selected semantic factors. We study a preceding problem as well: natural human specifications may leave actions, grounded entities, goals, or execution structure implicit. We therefore separate the need to *recover executable information* from the residual problem of expressing recovered information through the policy’s learned language interface. Frequencies in robot-training language provide a proxy for the latter.

C. Human-Centered Evaluation of Generalist Robot Policies.

Recent benchmarks increasingly move beyond aggregate task success by isolating visual, relational, procedural, and linguistic competencies. RoboLab probes controlled manipulation capabilities and instruction specificity [20]; LIBERO-Para targets paraphrase robustness [16]; and related suites evaluate linguistic and semantic generalization under controlled task variations [17], [18]. Our evaluation is complementary: we ask not only whether a policy can execute a physical objective, but how reliably that capability can be reached from the ways a non-expert naturally specifies it. For a fixed task, we characterize the *information gap* between a human specification and its executable structure, then measure the residual *alignment gap* once that structure is fully resolved. This separates failures to infer what the user means from failures to access an already learned behavior through the language or observation conditions expected by the policy.

III. METHOD

We study the communication gap by placing human and policy-facing language around a common representation of the physical task. Given a demonstrated objective, we first recover its grounded sequence of manipulation events. We then characterize how humans naturally specify these events and how the same events are expressed in robot-training language. This common task representation lets us measure, first, what executable structure a human specification leaves unresolved and, second, whether a fully resolved specification reaches the VLA through its learned language and observation interface.

A. Problem Formulation

We consider a physical task τ demonstrated by a video $V_\tau = (I_1, \dots, I_T)$, where I_t is the RGB observation at time t . From the demonstration, we recover an ordered sequence of grounded manipulation events

$$E_\tau = (e_1, \dots, e_{K_\tau}), \quad e_k = (A_k, O_k, D_k), \quad (1)$$

where A_k denotes the physical action, O_k the manipulated object, and D_k its destination or resulting target state. For example, placing a banana and an apple into a bowl yields two events, one for each object manipulation. E_τ serves as the common semantic reference throughout the study. Human specifications may selectively state, abstract, or compose information from these events, whereas robot-oriented commands express the same events using forms observed in robot-training data. Thus, all instruction variants preserve the same underlying physical objective while differing only in how that objective is communicated. For each task, let $\mathcal{H}\tau = \{u_\tau^1, \dots, u_\tau^N\}$ denote its human-like specifications, and let $\mathcal{R}\tau = \{r_\tau^1, \dots, r_\tau^M\}$ denote fully grounded robot-oriented realizations of the same event sequence.

Given a fixed VLA policy π , we evaluate the same task under these different specifications. The *information gap* is characterized by what structure in E_τ is not directly available from a human specification and must therefore be recovered for execution. After this structure is fully resolved, we separately probe the residual *execution gap*: whether the same executable task remains reliable as its events are organized differently for execution and invoked from different robot states. We recover E_τ from demonstration videos using object-centric tracking and persistent state and relation changes, with a vision-language model providing factual descriptions of the extracted transitions for subsequent language generation. Full implementation details are provided in Appendix B.

B. Characterizing Human Objective Specification

Natural objective specification is open-ended: people may describe an action, a desired outcome, or a higher-level intent; refer to entities explicitly or through the observed scene; and compress several manipulations into a single expression. Rather than defining this space a priori, we first study how people naturally instruct a robot from demonstrated behavior.

Human Instruction Elicitation. Participants observe task demonstrations of varying complexity and write the instruction they would give a robot to achieve the demonstrated objective. We do not prescribe a command format or require individual actions to be enumerated, allowing participants to decide what task information to state explicitly, how to identify relevant entities, and how to express compound objectives. These instructions provide both empirical evidence for constructing our taxonomy and a natural distribution against which its forms can be analyzed.

Deriving the Specification Taxonomy. We analyze the collected instructions together with distinctions from prior work on task-oriented and situated language [2], [4]–[6]. Three recurring factors capture the major ways specifications differ:

Factor	Form	Controlled variation	Example
Task content	Action	Action performed	“ Put all the yellow objects in the left bin and all the blue objects in the right bin.”
	Outcome	Desired physical goal state	“All yellow objects should be in the left bin, and all blue objects should be in the right bin.”
	Intent / preference	Desired effect or preference	“ I want the objects sorted by color. ”
Reference form	Direct	Directly named referent	“Put the yellow mug in the left bin and the blue cup in the right bin .”
	Grounding-dependent	Scene-resolved referent	“Put the yellow mug in the bin on the left and the blue cup in the bin on the right .”
Composition	Enumerated	One manipulation per clause	“Put the yellow mug in the left bin, then put the yellow bowl in the left bin, then put the blue cup in the right bin.”
	Grouped	Shared predicate across objects	“Put the yellow mug and bowl in the left bin, and the blue cup and plate in the right bin.”
	Collective	Set-level description	“Put all the yellow objects in the left bin and all the blue objects in the right bin.”

TABLE I: Factorized taxonomy of human objective specification. Each utterance may instantiate one form along each applicable factor. Highlighted spans illustrate the variation controlled by each form.

task content, *reference form*, and *composition* (Table I). The *task-content* factor captures whether the speaker states an action, a desired physical outcome, or an intent or preference from which the physical objective must be inferred. The *reference* factor captures whether relevant entities are named directly or identified through scene-dependent descriptions. The *composition* factor captures whether multiple manipulations are enumerated individually, grouped under a shared predicate, or compressed into a collective description. These factors are compositional rather than mutually exclusive: an outcome specification may use grounding-dependent references, while a collective specification may simultaneously compress multiple manipulations. The taxonomy serves two roles. First, we use it descriptively to code the collected human instructions and measure how the information people provide changes with task complexity. Second, we use the same factors experimentally to generate controlled specification variants for VLA evaluation, varying what task structure is exposed while keeping the underlying physical objective fixed.

Taxonomy-Conditioned Specification Generation. Given the grounded event sequence E_τ , we generate human-like specifications by selecting a configuration of the three taxonomy factors. A *content operator* determines whether the specification expresses the demonstrated action, its resulting physical state, or a higher-level intent. A *reference operator* determines whether objects and destinations are named directly or described through attributes, spatial relations, or semantic groupings available in the scene. A *composition operator* determines whether multiple events are expressed separately, grouped under a shared predicate, or summarized collectively. These variants preserve the same physical objective while changing how much event structure is made explicit. We then use a language model to produce lexical and syntactic variants without changing the selected taxonomy configuration, providing controlled specifications with known executable structure.

C. Characterizing the VLA Language Interface

The human-specification space characterizes how people communicate the same physical objective with different amounts and forms of task information. To characterize the other side of the interface, we model how fully resolved ma-

nipulation events are expressed in robot-training data. Unlike the human space, which varies what task structure is made explicit, the policy-facing space keeps the grounded task fixed and varies the linguistic forms through which the VLA was exposed to actions, objects, and destinations during training. This provides a proxy for the language interface through which resolved task information reaches the policy.

Robot-Training Language Prior. For each evaluated policy, we mine the robot-language dataset corresponding to its fine-tuning distribution. Following prior observations that robot-training language is substantially narrower and more repetitive than unrestricted human language [1], we extract recurring ways in which grounded manipulation events are expressed in the relevant training corpus. For each instruction, we extract its command template together with the action, manipulated-object reference, and destination reference. Grounded entities are replaced with typed placeholders to normalize commands with the same structure. These recurring forms provide a language prior for constructing policy-facing commands that are compatible with the policy’s fine-tuning distribution. Details of corpus processing and template normalization are provided in Appendix C.

Policy-Facing Realization. Given a grounded event $e_k = (A_k, O_k, D_k)$, we instantiate a compatible template mined from the policy’s fine-tuning data with its action, object, and destination to obtain a fully explicit policy-facing command r_k . For a compound task, $E_\tau = (e_1, \dots, e_{K_\tau})$, this yields an atomic command sequence $R_\tau = (r_1, \dots, r_{K_\tau})$. These realizations provide a standardized, fully resolved representation of the task in a form compatible with the policy’s training interface. They serve as the reference against which human specifications are compared and, for compound tasks, as the atomic commands used to study execution after the information gap has been resolved. We keep these realizations fixed in the controlled experiments so that performance differences reflect unresolved task information or execution conditions rather than incidental linguistic variation.

D. Measuring the Communication Gap

We characterize the *information gap* by comparing each specification against the grounded event sequence E_τ for the same physical objective. Because a single expression may

jointly describe several manipulations, we measure both what executable information is available and how explicitly the underlying events are decomposed, then relate these unresolved structures to VLA behavior. After the task is fully resolved, we separately measure the residual *execution gap* by varying how the same event sequence is organized for execution and the robot state from which each event is invoked.

Semantic Coverage and Grounding Dependence. For each event $e_k = (A_k, O_k, D_k)$, let $S_k = \{A_k, O_k, D_k\}$. For every required component $s \in S_k$, we assign $\sigma_{k,s}(u) \in \{0, 1, 2\}$, where $\sigma_{k,s}(u) = 0$ if the component is omitted from specification u , $\sigma_{k,s}(u) = 1$ if it is specified but its grounded value must be resolved from the scene or task context, and $\sigma_{k,s}(u) = 2$ if its grounded value is directly specified.

We define the *semantic coverage ratio*

$$\frac{\sum_{k=1}^{K_\tau} \sum_{s \in S_k} \mathbb{1}[\sigma_{k,s}(u) \geq 1]}{3K_\tau}, \quad (2)$$

which measures the fraction of required task components represented in the specification, either directly or through a grounding-dependent expression. We separately define the *groundedness ratio*

$$\frac{\sum_{k=1}^{K_\tau} \sum_{s \in S_k} \mathbb{1}[\sigma_{k,s}(u) = 2]}{3K_\tau}, \quad (3)$$

which measures the fraction whose grounded values are directly given in language. Thus, $1 - \eta_\tau(u)$ captures omitted task information, while $\eta_\tau(u) - \gamma_\tau(u)$ captures information that is present but still requires grounding.

Task Decomposition. Semantic coverage does not capture whether a compound objective is explicitly organized into its constituent manipulations. Let $J_\tau(u)$ denote the number of distinct manipulation units individuated in specification u . We define the decomposition ratio $\frac{J_\tau(u)}{K_\tau}$. A value of $\delta_\tau(u) = 1$ indicates that every atomic event is separately expressed, while smaller values indicate that multiple events are compressed into a shared expression.

Reasoning Required to Resolve the Specification. The same amount of unresolved structure may require qualitatively different forms of recovery. We additionally categorize specifications by the reasoning needed to map them onto E_τ , including visual grounding, action inference, task decomposition, and intent inference. We use this categorization as a complementary analysis of *what computation* is required to close the information gap; details are provided in Appendix G.

Information-Gap Evaluation. For each specification u , we execute the corresponding physical task with a fixed VLA policy and measure task success and partial progress. We compare it against a matched fully resolved realization r_τ^{full} constructed from the same event sequence E_τ . This reference uses Action–Direct–Enumerated semantics: every required action, object, and destination is directly specified, and every manipulation event is explicitly individuated. All events are presented together in a single prompt, so this comparison

resolves the task information. For evaluation metric M , we define the behavioral information gap as

$$\Delta_M^{\text{info}}(u) = M(r_\tau^{\text{full}}) - M(u). \quad (4)$$

A larger $\Delta_M^{\text{info}}(u)$ indicates a larger performance cost from executable structure left unresolved in u . We analyze this cost primarily as a function of semantic coverage $\eta_\tau(u)$, groundedness $\gamma_\tau(u)$, decomposition $\delta_\tau(u)$, and task complexity. A complementary analysis by required reasoning operation is provided in Appendix G. Together, these analyses characterize both *how much* executable structure is unresolved and *which kind* the policy struggles to recover.

Residual Execution Gap. We next ask whether resolving the information gap is sufficient for reliable execution. All conditions use the same fully specified Action–Direct–Enumerated event sequence and the same policy-facing atomic commands $R_\tau = (r_1, \dots, r_{K_\tau})$. We vary only how these commands are executed. First, we compare presenting all atomic events together in one prompt with issuing the same events sequentially, one command at a time. This isolates sensitivity to *execution organization* while holding the task semantics fixed. For metric M , we measure

$$\Delta_M^{\text{org}} = M(R_\tau^{\text{sequential}}) - M(R_\tau^{\text{one prompt}}). \quad (5)$$

Second, under sequential execution, we vary the robot state from which each subsequent command is invoked. In the chained condition, each command begins from the terminal robot configuration left by the preceding event. In the reset condition, the arm is returned to its home configuration before the next command while preserving the task state produced by previous events. We measure

$$\Delta_M^{\text{state}} = M(R_\tau^{\text{sequential + home reset}}) - M(R_\tau^{\text{sequential}}). \quad (6)$$

The home reset is not proposed as a solution, but serves as a generic intervention for testing sensitivity to invocation state. Together, these comparisons separate failures to recover the task from failures to execute it once fully specified. Full execution and evaluation details are provided in Appendix D.

IV. EXPERIMENTS

We investigate three questions about the human–VLA communication gap. **Q1:** How do natural human specifications change with task complexity, and what task information do people increasingly compress or leave implicit? **Q2:** Which forms of unresolved structure can the VLA recover, and how does their cost change as tasks become more complex? **Q3:** Once the task information is fully resolved, what residual alignment gap remains in the language realization and the observation from which the same learned behavior is invoked?

A. Experimental Setup

Environment and Policy. We evaluate a fixed $\pi_{0.5}$ policy on RoboLab’s 120 manipulation tasks [20], using 10 rollouts for each task–instruction condition. We report task success and normalized partial progress as the primary behavioral metrics. Across all comparisons, the underlying physical objective is

held fixed while only its specification or presentation to the policy is varied.

Human Instruction Study. To study how people naturally communicate physical objectives, we select 15 tasks spanning single-event manipulations and compound objectives with different structural regularities. Fifty participants view each demonstration without its canonical instruction and write the instruction they would naturally give a robot, yielding 750 specifications. We use these annotations to analyze how specification length, semantic availability, grounding dependence, and decomposition change with task complexity and structure.

Task Complexity and Compression Affordance. We measure physical task complexity by the number of grounded atomic events K_τ . Because tasks with the same event count may differ substantially in how naturally they can be compressed, we additionally distinguish compound tasks that require event-wise enumeration, admit a compact grounded rule, or admit a higher-level task effect. These labels are assigned from the grounded objective independently of the human-written specifications. Full definitions and the RoboLab-120 distribution are provided in Appendix A.

Controlled VLA Evaluation. For all 120 tasks, we generate human-like specifications under controlled taxonomy configurations and evaluate them against the matched fully grounded realization r_τ^G . This produces the factorial evaluation in Table II, where task content, reference form, and composition systematically vary the executable structure exposed to the policy. To study the residual alignment gap, we then hold the grounded event sequence fixed and vary how its atomic events are organized and when they are issued to the policy. Full execution protocols are provided in Appendix D.

Taxonomy and Generation Validation. The proposed taxonomy covers 96.7% of the naturally elicited specifications. Taxonomy-conditioned generations achieve 96.0% semantic correctness and are identified as human-written at approximately chance level in a matched source-identification study. Full taxonomy-coverage and generation-quality analyses are reported in Appendix E.

B. Humans Compress as Task Complexity Increases

We first ask whether natural human communication actually creates the information gap studied in Sec. III-D. If people expressed a compound objective by simply concatenating atomic commands, instruction length and explicit event structure should grow with the number of required manipulations. Instead, we find people increasingly compress the objective, leaving more executable structure implicit.

Instructions Remain Compact as Task Complexity Grows. Fig 2 compares natural specifications as the number of required atomic events K_τ increases. Instruction length grows only modestly, from 9.1 words for single-event tasks to 15.8 words for tasks requiring three or more events. At the same time, the executable structure made explicit declines substantially. Semantic coverage η_τ decreases from 0.95 to 0.74, while groundedness γ_τ falls from 0.89 to 0.27 and decomposition δ_τ from 1.00 to 0.21. Thus, people do not communicate

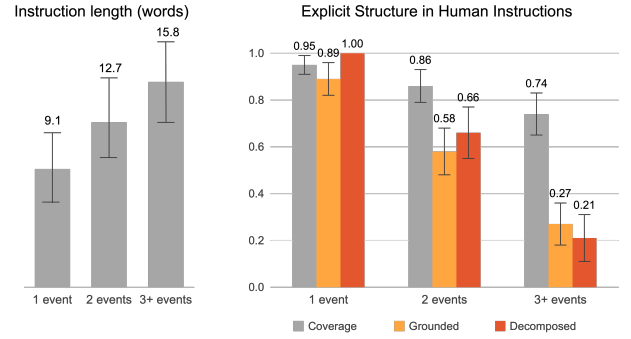


Fig. 2: Humans increasingly compress execution structure as tasks grow. Instruction length increases with the number of required events (left), but the executable structure made explicit decreases (right). Semantic coverage declines moderately, while groundedness and decomposition fall sharply, showing that people increasingly rely on contextual grounding and compress multiple manipulations rather than enumerating them. Error bars denote 95% confidence intervals.

increasingly complex objectives by proportionally increasing instruction length. They preserve much of the overall task content while increasingly relying on the listener to resolve referents and, especially, to expand a compact expression into the required atomic manipulations. Natural communication therefore produces a growing information gap precisely as the physical objective becomes more compositional.

Compression Exploits Available Task Structure. This compression is not simply a consequence of having more events. Tasks with the same K_τ differ in whether their events admit a compact shared description, and people exploit this structure when it is available. Weakly structured compound objectives remain comparatively action-oriented and enumerative, whereas rule-compressible objectives shift toward outcome-level and collective descriptions. When the task admits a compact semantic effect, participants more often describe the intent itself and leave the concrete physical realization implicit. The full distribution across task-content and composition forms is reported in Appendix F. Together, these results show that human compression is *structure-dependent*: people preserve event-wise detail when the objective requires it, but use increasingly abstract specifications when a compact rule or intent can characterize the same physical goal.

C. VLAs: Selectively Sensitive to Unresolved Task Structure

Humans leave different kinds of executable structure unresolved. We next ask whether the VLA can recover them equally well. Table II evaluates the full factorial space of taxonomy-controlled specifications against a matched fully grounded realization of the same physical objective.

What matters is not simply how much information is unresolved, but what must be recovered. The factorial design isolates each taxonomy factor while holding the others maximally explicit. Along the task-content axis, replacing an action specification with an outcome specification increases the matched success gap by only 4.5 percentage points, whereas an intent-level specification increases it to 15.2 points. An explicit goal state therefore often constrains the missing action sufficiently for the policy to recover it, while an intent

Content	Taxonomy configuration		Available executable structure			VLA behavior			N
	Reference	Composition	η_τ	γ_τ	δ_τ	Success (%)	Progress	$\Delta_{\text{Success}}^{\text{info}}$	
Action	Direct	Enumerated	1.00	1.00	1.00	38.0 $\pm$ 5.4	0.51 $\pm$ 0.05	0.0 (ref.)	120
		Grouped	1.00	1.00	0.47	15.9 $\pm$ 6.0	0.39 $\pm$ 0.06	7.6 $\pm$ 3.1	54
		Collective	1.00	0.67	0.41	13.1 $\pm$ 5.8	0.35 $\pm$ 0.06	10.4 $\pm$ 3.5	50
	Grounding-dep.	Enumerated	1.00	0.50	1.00	35.2 $\pm$ 5.3	0.48 $\pm$ 0.05	2.8 $\pm$ 1.8	120
		Grouped	1.00	0.50	0.47	13.5 $\pm$ 5.6	0.35 $\pm$ 0.05	10.0 $\pm$ 3.3	54
		Collective	1.00	0.50	0.41	12.4 $\pm$ 5.7	0.34 $\pm$ 0.05	11.1 $\pm$ 3.6	50
	Missing	Enumerated	0.33	0.33	1.00	26.9 $\pm$ 4.9	0.38 $\pm$ 0.05	11.1 $\pm$ 2.3	120
		Grouped	0.33	0.33	0.47	7.1 $\pm$ 4.2	0.24 $\pm$ 0.04	16.4 $\pm$ 3.9	54
		Collective	0.33	0.33	0.00	6.5 $\pm$ 4.2	0.23 $\pm$ 0.04	17.0 $\pm$ 4.1	50
Outcome	Direct	Enumerated	0.67	0.67	1.00	33.5 $\pm$ 5.2	0.46 $\pm$ 0.05	4.5 $\pm$ 1.9	120
		Grouped	0.67	0.67	0.47	11.8 $\pm$ 5.3	0.33 $\pm$ 0.05	11.7 $\pm$ 3.5	54
		Collective	0.67	0.33	0.41	9.7 $\pm$ 5.1	0.29 $\pm$ 0.05	13.8 $\pm$ 3.8	50
	Grounding-dep.	Enumerated	0.67	0.17	1.00	31.6 $\pm$ 5.2	0.44 $\pm$ 0.05	6.4 $\pm$ 2.0	120
		Grouped	0.67	0.17	0.47	10.0 $\pm$ 5.0	0.30 $\pm$ 0.05	13.5 $\pm$ 3.7	54
		Collective	0.67	0.17	0.41	9.2 $\pm$ 5.0	0.28 $\pm$ 0.05	14.3 $\pm$ 3.9	50
	Missing	Enumerated	0.00	0.00	1.00	24.6 $\pm$ 4.8	0.35 $\pm$ 0.05	13.4 $\pm$ 2.4	120
		Grouped	0.00	0.00	0.47	5.3 $\pm$ 3.7	0.20 $\pm$ 0.04	18.2 $\pm$ 4.1	54
		Collective	0.00	0.00	0.00	4.9 $\pm$ 3.7	0.19 $\pm$ 0.04	18.6 $\pm$ 4.3	50
Intent	Direct	Enumerated	0.33	0.33	1.00	22.8 $\pm$ 4.7	0.33 $\pm$ 0.04	15.2 $\pm$ 2.6	120
		Grouped	0.33	0.33	0.47	5.3 $\pm$ 3.7	0.20 $\pm$ 0.04	18.2 $\pm$ 4.1	54
		Collective	0.33	0.00	0.41	4.3 $\pm$ 3.5	0.18 $\pm$ 0.03	19.2 $\pm$ 4.4	50
	Grounding-dep.	Enumerated	0.33	0.00	1.00	22.1 $\pm$ 4.6	0.32 $\pm$ 0.04	15.9 $\pm$ 2.6	120
		Grouped	0.33	0.00	0.47	4.7 $\pm$ 3.5	0.19 $\pm$ 0.03	18.8 $\pm$ 4.2	54
		Collective	0.33	0.00	0.41	4.3 $\pm$ 3.5	0.18 $\pm$ 0.03	19.2 $\pm$ 4.4	50
	Missing	Enumerated	0.00	0.00	1.00	18.8 $\pm$ 4.3	0.27 $\pm$ 0.04	19.3 $\pm$ 2.8	120
		Grouped	0.00	0.00	0.47	3.2 $\pm$ 2.9	0.15 $\pm$ 0.03	20.3 $\pm$ 4.3	54
		Collective	0.00	0.00	0.00	2.9 $\pm$ 2.9	0.14 $\pm$ 0.03	20.6 $\pm$ 4.5	50

TABLE II: VLA performance across the full factorial space of taxonomy-controlled specifications. Content and reference forms are generated for all 120 tasks. Enumerated composition applies to all tasks, while grouped and collective forms are evaluated on 54 and 50 eligible tasks, respectively. Results are first averaged over 10 rollouts and multiple surface realizations for each task-configuration pair, and then across eligible tasks. Behavioral results are reported as mean $\pm$ 95% confidence-interval half-width, computed over task-level means. η_τ , γ_τ , and δ_τ denote semantic coverage, groundedness, and explicit execution structure, respectively. $\Delta_{\text{Success}}^{\text{info}}$ is a matched task-level difference relative to r_τ^G over the same eligible task subset.

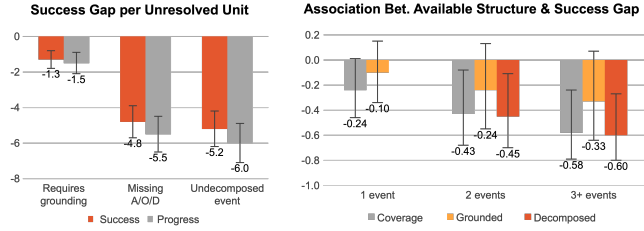


Fig. 3: Behavioral cost of unresolved task structure. Left: Estimated performance change from one additional unresolved unit. Grounding-dependent information incurs a small penalty, while missing action/object/destination information and undecomposed events cause substantially larger, comparable losses. Right: Correlation between available executable structure and the success gap across task complexity. Semantic coverage and decomposition become increasingly predictive for longer tasks, whereas groundedness remains weaker. Error bars denote 95% confidence intervals.

leaves the policy to infer the physical goal itself, together with the actions needed to realize it. The same distinction appears in reference. With action content and enumerated composition fixed, grounding-dependent references incur only a 2.8 point gap, while omitting the relevant reference information increases the gap to 11.1 points. The policy is therefore comparatively robust when the needed information is present but must be resolved from the scene; it is much less reliable when that information is absent and must be reconstructed.

Missing event structure is as consequential as missing task semantics. With action content and direct references fixed, grouping multiple manipulations produces a 7.6 point success gap, and collective specification increases the gap to 10.4 points. These losses are comparable in scale to those caused by genuinely missing semantic components. The problem is therefore not only semantic omission: even when the goal and relevant entities are available, compressing several manipulations into one expression forces the policy

to infer the latent event structure needed for execution. The aggregate analysis in Fig. 3 makes this distinction clearer. An additional grounding-dependent component has only a small behavioral cost, whereas a missing action, object, or destination and an undecomposed manipulation event incur substantially larger and comparable penalties. This separates two regimes of recovery: *contextual resolution*, where language and observation already constrain the missing value, and *structural reconstruction*, where the policy must infer task semantics or execution structure that is not explicitly available.

Structural reconstruction becomes harder as tasks grow.

As task complexity increases, both semantic coverage η_τ and decomposition δ_τ become increasingly associated with the success gap, while groundedness γ_τ remains a weaker predictor. Thus, longer tasks do not merely accumulate more room for error. They amplify the cost of reconstructing omitted semantics and, especially, the event structure hidden by compressed specifications. This produces the central human-VLA mismatch. People increasingly rely on contextual grounding and compress event structure as tasks become more complex (Fig. 2). The former is relatively well tolerated by the policy; the latter moves the specification into the same costly regime as genuinely missing task semantics. Human communication therefore becomes more compact by shifting responsibility to precisely the kind of structural inference that the VLA is least reliable at performing.

D. Alignment Gap After Information Resolution

The preceding experiments characterize what executable information the policy needs. We next ask whether providing that information completely is sufficient for reliable execution. We fix every compound task to the same Action-Direct-

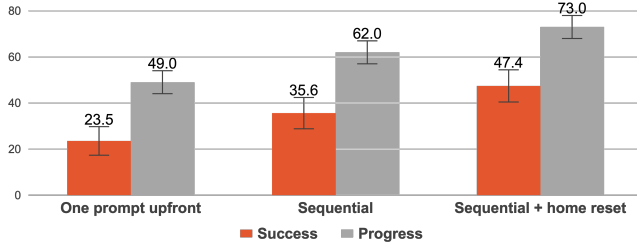


Fig. 4: Residual Alignment Gap: effect of command organization and invocation state. All conditions use the same fully specified Action-Direct-Enumerated task semantics. Issuing the same atomic events sequentially improves performance over presenting them together in one prompt, showing sensitivity to command organization. Resetting the arm to its home configuration between events yields a further gain, revealing additional sensitivity to the observation from which each skill is invoked.

Enumerated specification, so that all required actions, objects, destinations, and atomic events are explicit, and vary only how these resolved events are presented and invoked.

Sequentializing Resolved Events Substantially Improves Execution. As shown in Fig. 4, presenting all atomic events together in a single prompt achieves 23.5% success on compound tasks. Issuing the same fully grounded events sequentially, one at a time, raises success to 35.6%. No additional task information is introduced: both conditions specify the same actions, objects, destinations, and event decomposition. The difference is instead in execution structure. A long enumerated prompt requires the policy to maintain and execute several manipulations from a single instruction, whereas sequentialization reduces each invocation to one grounded atomic event. For policies trained largely on short, local manipulation commands, the former can fall outside the execution structure encountered during training. Thus, even after the information gap is removed, the resolved objective must be presented at a granularity the policy can reliably execute.

A Familiar Invocation State Further Improves Reliability. Sequential execution still invokes each later event from the terminal robot state produced by the preceding manipulation. Returning the arm to its home configuration before each subsequent command further raises success from 35.6% to 47.4%. This comparison holds the active atomic command and task-relevant object configuration fixed while changing only the robot configuration from which the next skill is invoked. The resulting gain exposes a residual *observation-alignment gap*: even when the correct atomic behavior is specified at the appropriate granularity, access to that behavior remains sensitive to the state from which it is invoked.

V. CONCLUSION

We identify a human-VLA communication gap between how people naturally specify tasks and how current policies can reliably act on them. Beyond diagnosing this gap, our analysis provides a way to localize where communication breaks down and what structure must be recovered, offering concrete targets for both policy learning and interface design. We hope this perspective can guide future work toward robots that better interpret naturally expressed objectives and interfaces that bridge human intent to executable behavior. Ultimately,

generalist robots should adapt to how people communicate, rather than requiring people to adapt to the robot’s learned interface.

REFERENCES

- [1] S. L. Wanda *et al.*, “Limited linguistic diversity in embodied AI datasets,” in *Proc. 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2026, pp. 11 058–11 086.
- [2] J. R. Searle, “Indirect speech acts,” in *Syntax and Semantics, Volume 3: Speech Acts*, P. Cole and J. L. Morgan, Eds. Academic Press, 1975, pp. 59–82.
- [3] H. H. Clark and D. H. Schunk, “Polite responses to polite requests,” *Cognition*, vol. 8, no. 2, pp. 111–143, 1980.
- [4] B. Di Eugenio, “Understanding natural language instructions: The case of purpose clauses,” in *Proc. 30th Annual Meeting of the Association for Computational Linguistics*, 1992, pp. 120–127.
- [5] K. Vander Linden and B. Di Eugenio, “A corpus study of negative imperatives in natural language instructions,” in *Proc. 16th International Conference on Computational Linguistics (COLING)*, 1996.
- [6] H. H. Clark and D. Wilkes-Gibbs, “Referring as a collaborative process,” *Cognition*, vol. 22, no. 1, pp. 1–39, 1986.
- [7] E. Bastianelli *et al.*, “HuRIC: A human robot interaction corpus,” in *Proc. 9th International Conference on Language Resources and Evaluation (LREC)*, 2014, pp. 4519–4526.
- [8] S. Zarriß *et al.*, “PentoRef: A corpus of spoken references in task-oriented dialogues,” in *Proc. 10th International Conference on Language Resources and Evaluation (LREC)*, 2016, pp. 125–131.
- [9] S. M. Lukin *et al.*, “SCOUT: A situated and multi-modal human-robot dialogue corpus,” in *Proc. Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING)*, 2024, pp. 14 445–14 458.
- [10] A. Brohan *et al.*, “RT-1: Robotics transformer for real-world control at scale,” in *Proc. Robotics: Science and Systems (RSS)*, 2023.
- [11] H. R. Walke *et al.*, “BridgeData V2: A dataset for robot learning at scale,” in *Proc. 7th Conference on Robot Learning (CoRL)*, 2023, pp. 1723–1736.
- [12] E. Rosete-Beas *et al.*, “Latent plans for task-agnostic offline reinforcement learning,” in *Proc. 6th Conference on Robot Learning (CoRL)*, 2023, pp. 1838–1849.
- [13] C. Lynch *et al.*, “Interactive language: Talking to robots in real time,” *IEEE Robotics and Automation Letters*, 2023, early access, doi: 10.1109/LRA.2023.3295255.
- [14] B. Liu *et al.*, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [15] M. Shridhar *et al.*, “ALFRED: A benchmark for interpreting grounded instructions for everyday tasks,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 10 740–10 749.
- [16] C. Kim *et al.*, “LIBERO-Para: A diagnostic benchmark and metrics for paraphrase robustness in VLA models,” *arXiv preprint arXiv:2603.28301*, 2026.
- [17] Y. Hou and L. Zhao, “LangGap: Diagnosing and closing the language gap in vision-language-action models,” *arXiv preprint arXiv:2603.00592*, 2026.
- [18] I. Fang *et al.*, “From intention to execution: Probing the generalization boundaries of vision-language-action models,” *arXiv preprint arXiv:2506.09930*, 2025.
- [19] Z. Yin and Z. Zhang, “Grounded semantic re-binding for robust instruction generalization in vision-language-action models,” *arXiv preprint arXiv:2608.02497*, 2026.
- [20] X. Yang *et al.*, “RoboLab: A high-fidelity simulation benchmark for analysis of task generalist policies,” in *Proc. Robotics: Science and Systems (RSS)*, 2026.
- [21] T. Ren *et al.*, “Grounding DINO 1.5: Advance the “edge” of open-set object detection,” *arXiv preprint arXiv:2405.10300*, 2024.
- [22] N. Ravi *et al.*, “SAM 2: Segment anything in images and videos,” *arXiv preprint arXiv:2408.00714*, 2024.
- [23] L. Yang *et al.*, “Depth anything V2,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [24] S. Bai *et al.*, “Qwen2.5-VL technical report,” *arXiv preprint arXiv:2502.13923*, 2025.

APPENDIX

A. Task-Structure Complexity and Compression Affordance

Our analysis distinguishes the *number of required physical events* from the *structure available for expressing those events compactly*. Two tasks with the same number of manipulations may afford very different specifications: unrelated manipulations may require event-wise enumeration, whereas several placements may be summarized by a rule such as “put all the black objects in the bin.”

a) *Atomic Event Count.*: For each task τ , we use the grounded event sequence

$$E_\tau = (e_1, \dots, e_{K_\tau}), \quad (7)$$

where each $e_k = (A_k, O_k, D_k)$ denotes one object-centric physical transition. K_τ therefore measures physical manipulation complexity at the same granularity used throughout our information-gap analysis. It is derived from the extracted demonstration rather than from instruction length or linguistic structure.

This differs from RoboLab’s internal `num_subtasks` statistic, which is defined by the benchmark’s success-condition representation [20]. We use the latter only as auxiliary metadata and compute K_τ from E_τ .

b) *Compression Affordance.*: Event count alone does not determine how compactly an objective can be communicated. For tasks with $K_\tau > 1$, we classify the strongest task-level structure that permits the objective to be expressed without enumerating every event. Classification is performed from E_τ and the corresponding grounded scene, independently of the human-written specifications.

Enumerative tasks require event-specific object, destination, or ordering information to remain explicit. Even when events share a generic action or destination, no single non-enumerative rule fully determines the required event sequence.

Rule-compressible tasks admit a grounded rule P_τ over observable entities, attributes, relations, or target states such that $E_\tau = \mathcal{E}(P_\tau, G_\tau)$, where $\mathcal{E}$ instantiates the atomic events satisfying P_τ in scene G_τ . Examples include ‘all black objects in the bin,’ ‘all mugs upright,’ or sorting objects by color.

Intent-compressible tasks admit a higher-level semantic effect or purpose F_τ that compactly characterizes the objective without directly specifying the grounded object–destination assignments. Recovering the executable task therefore requires intent-to-goal inference: $E_\tau = \mathcal{I}(F_\tau, G_\tau)$, where $\mathcal{I}$ maps the task-level effect and current scene to the corresponding grounded events. Examples include clearing a work surface, packing objects for storage, or discarding unwanted items. Table III summarizes the operational criteria.

c) *RoboLab-120 Distribution.*: We apply this coding scheme to the 120 RoboLab tasks [20]. Table IV reports the resulting distribution.

Nearly half of RoboLab tasks require multiple object-centric manipulations, but most compound objectives also exhibit reusable structure. RoboLab therefore contains many cases in which physical complexity increases while the objective

remains linguistically compressible, allowing us to study whether people exploit this structure when deciding what to state explicitly and what to leave for the listener to recover.

B. Grounded Event Extraction

Our analysis assumes a common grounded event representation E_τ for each demonstrated task. Here we describe the perception and verbalization pipeline used to recover this representation from demonstration videos. This component provides a shared semantic source for human-specification generation and policy-facing realization; it is not itself the focus of our study.

a) *Object-Centric Temporal Representation.*: Given a demonstration $V_\tau = (I_1, \dots, I_T)$, we convert the video into a sequence of object-centric scene graphs. We use Grounding DINO 1.5 Edge for open-vocabulary object detection [21] and SAM 2 for mask propagation and persistent object tracking [22]. Grounding DINO is applied to initialize object tracks and to recover objects when tracks are lost, while SAM 2 propagates object masks across intermediate frames.

Each frame is represented as a graph $G_t = (\mathcal{V}_t, \mathcal{E}_t)$. Each node stores a persistent object identity together with its category, segmentation mask, bounding box, visual attributes, approximate position, and state. Edges encode spatial and semantic relations between entities, including left/right, front/behind, near, on, and inside. Image-plane relations are computed from object masks, while relative depth is estimated using Depth Anything V2 [23]. The resulting graph sequence maintains consistent entities and relations across the demonstration.

b) *Event Extraction.*: We identify task-relevant events from persistent changes in object states and relations. A candidate change is retained only when the new state or relation remains stable across subsequent frames, reducing transient errors from tracking or segmentation. For the k -th transition, we define $\Delta G_k = G_{t_k^+} \ominus G_{t_k^-}$, where t_k^- and t_k^+ denote representative observations immediately before and after the transition. Consecutive changes involving the same manipulated entity are grouped into a single atomic event.

This yields an ordered event sequence $E_\tau = (e_1, \dots, e_{K_\tau})$, where each event is represented as $e_k = (A_k, O_k, D_k)$. Here, A_k denotes the manipulation, O_k the manipulated object, and D_k its destination or resulting target state. For example, moving a white mug from the table into a blue bowl produces an event of the form (place, white mug, inside(blue bowl)).

c) *Factual Event Verbalization.*: The structured event representation is used as the semantic source of truth, but directly exposing graph serialization to a language model makes downstream generation unnecessarily dependent on implementation details. We therefore additionally convert each event into a concise factual description.

For each transition, Qwen2.5-VL [24] receives the graph change ΔG_k together with observations immediately before and after the event. It is prompted to identify which entity changed, what changed, and the resulting state or relation. The response is constrained to entities and relations represented

Task structure	Operational criterion	Information retained by compact form	RoboLab examples
Atomic	$K_\tau = 1$	One grounded (A, O, D) event	BananaInBowlTask; BowlInBinTask
Compound, enumerative	$K_\tau > 1$ and critical event-specific arguments or ordering must still be enumerated	Individual object identities and/or event ordering	RubiksCubeAndBananaTask; BananaThenRubiksCubeTask; RubiksCubeThenBananaTask
Compound, rule-compressible	$K_\tau > 1$ and a grounded rule P_τ generates the required events without event-wise enumeration	Observable set membership, relation, shared state, or attribute-destination mapping	BlackItemsInBinTask; FoodPackingByColorTask; ReorientAllMugsTask; BlockStackingSpecifiedOrderTask
Compound, intent-compressible	$K_\tau > 1$ and a semantic effect F_τ compactly characterizes the objective while leaving concrete event structure to be inferred	Desired purpose or task-level effect rather than complete grounded (O, D) assignments	CleanUpToysTask; CookingClearPlateTask; RecycleCartonTask; ThrowAwaySnacksTask

TABLE III: Operational definition of task structure used to analyze human objective specification. Event count measures physical manipulation complexity, whereas compression affordance characterizes the structure available for expressing those events compactly.

Task structure	Tasks	Fraction (%)
Atomic	64	53.3%
Compound, enumerative	3	2.5%
Compound, rule-compressible	42	35.0%
Compound, intent-compressible	11	9.2%
Total	120	100.0%

TABLE IV: Structural distribution of RoboLab-120 under our object-centric event and compression-affordance definitions.

in ΔG_k . For example, a transition from $\text{ON}(\text{mug}, \text{table})$ to $\text{IN}(\text{mug}, \text{bowl})$ may be verbalized as “The white mug is moved from the table into the blue bowl.”

These descriptions serve only as a natural-language interface to the downstream specification generators. The grounded event record E_τ , rather than the generated description, determines the physical semantics of the task.

C. Robot-Training Language Mining

This appendix describes how we extract the policy-facing language distribution used in Sec. III-C. The goal is to characterize recurring ways in which grounded manipulation events are expressed in robot-training data, while preserving their empirical prevalence.

a) Corpus Construction.: We collect language annotations from RT-1 [10], BridgeData V2 [11], TacoPlay [12], Language Table [13], and LIBERO [14]. Each instruction is parsed into the task components corresponding to our grounded event representation: the action A , manipulated object O , and destination or resulting target state D .

b) Template Normalization.: To separate command structure from task-specific entities, spans corresponding to grounded task arguments are replaced with typed placeholders. For example, “Put the red mug inside the blue bowl” is normalized to “Put [OBJECT] inside [DESTINATION].” Instructions with the same normalized structure are grouped under a shared template. For each dataset d , we estimate the empirical template frequency as $p_d(t) = \frac{N_d(t)}{\sum_{t'} N_d(t')}$, where $N_d(t)$ is the number of instructions instantiating template t . We retain dataset-specific frequencies rather than treating all observed templates as equally likely.

c) Lexical Realization.: Within each template, we additionally record the lexical forms used to express grounded task

components. This includes alternative action expressions, such as ‘pick up,’ ‘grasp,’ and ‘lift,’ as well as alternative relational expressions for target states or destinations. For each semantic component, we retain the empirical frequency of its observed realizations.

d) Reference Forms.: We characterize how manipulated objects and destinations are identified in robot-training instructions. Reference forms include category names, attribute-qualified names, attribute-only expressions, and relational descriptions. For example, the same grounded entity may be referred to as ‘the mug,’ ‘the yellow mug,’ ‘the yellow object,’ or ‘the object beside the plate.’ We retain both the form and its empirical frequency in the source corpus.

e) Policy-Facing Realization Generation.: Given a grounded event $e_k = (A_k, O_k, D_k)$, we first restrict the mined template set to structures compatible with the event. We then instantiate a selected template using lexical and reference realizations observed for the corresponding task components. When a selected reference form requires visual attributes or spatial relations, these are obtained from the grounded scene representation associated with the task.

Alternative policy-facing realizations are produced by varying the compatible template, lexical realization, or reference form while preserving the same grounded event. For compound tasks, this procedure is applied independently to each event in $E_\tau = (e_1, \dots, e_{K_\tau})$, yielding an atomic policy-facing command sequence $R_\tau = (r_1, \dots, r_{K_\tau})$.

Each realization is stored together with the frequencies of the template, lexical choices, and reference forms from which it was constructed. These quantities are used as measures of training-language exposure in the language-alignment analysis.

D. Execution and Evaluation Protocol

This section provides the execution details for the information- and alignment-gap evaluations. Within each task, all compared conditions use the same physical objective, initial scene configuration, evaluation criteria, and total execution budget.

a) Behavioral Metrics.: We report task success and normalized partial progress. *Success* is determined by the bench-

mark success predicate for the corresponding task. *Progress* measures the fraction of the task objective completed when a compound task is only partially successful. Both metrics are used for the information- and alignment-gap analyses, with $M \in \text{Success, Progress}$.

b) Compound Specification Execution.: For human-like specifications and the fully grounded reference r_τ^G , the complete specification is provided at the beginning of the rollout and remains fixed during execution. The policy acts until the task succeeds or the execution budget is exhausted.

The reference r_τ^G explicitly specifies the required actions, manipulated objects, and destinations from E_τ , while preserving the objective as a single compound specification. It therefore resolves missing task semantics and grounding without separately controlling the temporal execution of individual events.

c) Atomic Sequential Execution.: For sequential conditions, the event sequence $E_\tau = (e_1, \dots, e_{K_\tau})$ is realized as atomic policy-facing commands $R_\tau = (r_1, \dots, r_{K_\tau})$. The policy first receives r_1 and advances to r_{k+1} when the completion predicate for e_k is satisfied or the allotted execution budget for the current event expires.

All presentation conditions use the same total execution budget, so sequential issuance changes how the task is presented rather than providing additional interaction time.

d) Observation-Alignment Intervention.: We compare two sequential execution conditions. In the *chained* condition, command r_{k+1} is issued directly from the terminal state produced by r_k . In the *home-reset* condition, the robot arm is returned to its home configuration before r_{k+1} is issued, while the task-relevant object configuration is preserved.

The atomic command and task semantics remain unchanged. The reset therefore provides a generic intervention on the invocation observation rather than modifying the intended behavior, allowing us to measure sensitivity to the state from which a learned skill is accessed.

e) Prompt-Presentation Baselines.: For compound objectives, we additionally compare a single grounded compound specification with a prompt containing all grounded atomic commands simultaneously. These conditions distinguish explicit event decomposition from sequential command issuance.

f) Language-Alignment Evaluation.: To evaluate language alignment, we hold the grounded task semantics fixed and construct alternative policy-facing realizations that vary command template, action lexicalization, object reference, or destination realization. Each realization is associated with its empirical frequency in the robot-language corpus described in Appendix C. The task configuration and rollout protocol are otherwise held fixed across matched realizations.

g) Matched Evaluation.: Each task–instruction condition is evaluated over 10 rollouts. Comparisons within a task use matched physical objectives and evaluation criteria, so differences in behavior arise from changes in the information or interface presented to the policy rather than from changes in the intended task.

Factor	Agreement (%)	Coverage (%)
Task content	91.3	98.0
Reference form	96.0	99.3
Composition	93.3	98.7
Complete specification	–	96.7

TABLE V: Coverage of naturally elicited specifications by the proposed taxonomy and agreement between independent annotators.

Task structure	Source ID. (%)	Correctness (%)
Atomic	50.8%	98.5%
Compound, enumerative	51.9%	97.3%
Compound, rule-compressible	53.7%	95.4%
Compound, intent-compressible	57.2%	90.6%
Overall	52.8%	96.0%

TABLE VI: Human-source identification and semantic correctness of taxonomy-matched generated specifications. Chance source identification is 50%.

E. Human-Specification and Generation Validation

We use the elicited human instructions both to characterize natural objective specification and to validate that the proposed taxonomy captures the forms people actually use. We separately evaluate whether taxonomy-conditioned generation preserves the intended task semantics while producing natural linguistic realizations.

a) Taxonomy Coverage.: Two independent annotators label each of the 150 elicited specifications along the three taxonomy factors: task content, reference form, and composition. A specification is considered covered when all task-relevant content can be represented by the proposed factors without introducing an additional *Other* category.

Table V reports factor-wise agreement and coverage.

The taxonomy covers 96.7% of complete specifications. The remaining cases are primarily discourse-dependent expressions, such as “do the same with the rest,” rather than recurring objective-specification strategies absent from the taxonomy.

b) Matched Human-Likeness Evaluation.: To evaluate whether taxonomy-conditioned generation reproduces natural realizations rather than merely valid taxonomy instances, we construct matched human–generated pairs. For each human annotation, we retain its task and taxonomy configuration and generate one specification with the same task-content, reference, and composition choices.

Twenty additional participants complete a blinded source-identification study in which they choose which member of each matched pair was written by a human. Because the pair is matched on the information communicated, the study evaluates surface realization rather than differences in semantic content.

c) Semantic Fidelity.: We separately evaluate whether generated specifications remain consistent with the grounded task objective. A held-out vision-language model compares each generated specification against the corresponding event sequence E_τ and determines whether the stated objective is semantically supported by the demonstration.

Table VI reports source-identification and semantic-correctness results by task structure.

Overall source-identification accuracy is 52.8%, close to the 50% chance level, while semantic correctness is 96.0%. Generation remains highly faithful for atomic and enumerative objectives and becomes somewhat more difficult for increasingly abstract task structures, particularly intent-compressible objectives. These results support using taxonomy-conditioned generation to construct controlled human-like specifications for the VLA evaluation.

F. Human Specification Strategies by Task Structure

The main text shows that natural specifications become less grounded and less explicitly decomposed as task complexity increases. Here we examine *how* people achieve that compression by analyzing the taxonomy forms they select for tasks with different structural affordances.

We group tasks according to the compression-affordance classes defined in Appendix A: single atomic events, compound objectives that remain primarily enumerative, compound objectives that admit a compact grounded rule, and compound objectives that admit a compact task-level intent. For each elicited specification, we record its task-content form, reference form, and composition form. These labels are assigned from the specification itself, while the task-structure class is assigned independently from the grounded objective.

Table VII reports the resulting distributions.

a) *Task Content.*: Single-event tasks are predominantly expressed as direct actions, with 82% of specifications using action-level descriptions. Compound objectives that do not admit a strong compact abstraction remain similarly action-oriented, with 68% expressed at the action level. By contrast, when a grounded rule captures the shared structure of the task, participants more often describe the desired outcome: outcome-level specifications account for 58% of rule-compressible tasks. Intent-compressible objectives shift further toward abstraction, with 55% expressed through an intent or higher-level effect.

This progression suggests that speakers do not choose task-content form independently of the objective structure. Concrete shared regularities favor outcome-level descriptions, whereas objectives naturally characterized by a purpose or effect invite intent-level specification.

b) *Reference Form.*: Reference strategies shift in parallel. For atomic tasks, 92% of specifications use direct references to the relevant objects or destinations. Direct reference remains dominant for compound enumerative tasks at 78%, but falls to 46% for rule-compressible tasks and 39% for intent-compressible tasks. As a result, increasingly compressible objectives place more responsibility on the listener to resolve the intended entities from scene context, set membership, or the inferred task goal.

c) *Composition.*: The strongest shift appears in how multiple manipulations are linguistically organized. Among compound enumerative tasks, 63% of specifications explicitly enumerate the required manipulations. When a shared grounded rule is available, only 12% remain enumerated, while 57% use a collective description. Intent-compressible tasks

show an even stronger preference for collective specification, at 69%.

Thus, event count alone does not explain whether speakers enumerate or compress a compound objective. Participants preserve event-wise structure when the task requires it, but collapse multiple manipulations into a single expression when the grounded objective supports a shared rule or higher-level effect.

d) *Implication for the Information Gap.*: The three taxonomy factors therefore shift together rather than independently. As task structure becomes more compressible, specifications move from action-level to outcome- or intent-level content, from direct to grounding-dependent reference, and from enumeration to collective composition. This produces a coordinated transfer of responsibility from speaker to listener: the speaker communicates a compact characterization of the desired objective, while the listener must recover the grounded entities, atomic manipulations, and, for intent-level descriptions, the physical goal itself.

This analysis complements the complexity result in Sec. IV-B. Increasing K_τ creates more opportunity for compression, but whether speakers actually exploit that opportunity depends on the structure of the objective. Natural human specification is therefore shaped jointly by physical complexity and by the compact abstractions the task affords.

G. Reasoning Requirements and VLA Reliability

The main analysis measures the information gap through the executable structure available in a specification: semantic coverage, groundedness, and event decomposition. Here we provide a complementary view based on the *reasoning required* to recover the unresolved structure.

a) *Reasoning Operations.*: We distinguish four recurring forms of recovery, summarized in Table VIII. *Visual grounding* resolves a referenced object or destination from the observed scene. *Action inference* recovers an omitted manipulation from an explicitly specified object and desired state. *Task decomposition* expands a compressed objective into its constituent grounded events. *Intent inference* first recovers the intended physical goal from a higher-level purpose or preference, and may subsequently require grounding, action inference, and decomposition.

These categories are not mutually exclusive: a single specification may require several operations. To estimate their behavioral cost, we therefore construct matched conditions that isolate one focal reasoning requirement while preserving the remaining task information whenever possible. Table IX reports success, partial progress, and the success gap relative to the matched fully resolved realization r_τ^{full} .

b) *Local Recovery Is Comparatively Reliable.*: Visual grounding produces a 4.6% success gap relative to the fully resolved reference, while action inference produces a 6.2% gap. In both cases, the unresolved component is strongly constrained by the remaining specification and observation: a contextual reference can often be resolved from the visible

Task structure	Task content			Reference form		Composition		
	Action	Outcome	Intent	Direct	Grounding-dep.	Enumerated	Grouped	Collective
Single atomic event	82%	16%	2%	92%	8%	–	–	–
Compound, enumerative	68%	27%	5%	78%	22%	63%	29%	8%
Compound, rule-compressible	34%	58%	8%	46%	54%	12%	31%	57%
Compound, intent-compressible	24%	21%	55%	39%	61%	9%	22%	69%

TABLE VII: Human specification strategies as a function of task structure. Percentages sum to 100% within each taxonomy factor. Reference form is computed over specifications containing an explicit referential expression, and composition is reported only for compound tasks.

Reasoning	Example	Unresolved structure	Required recovery
Visual grounding	“Pick up the leftmost cup.”	Grounded O_k or D_k	Resolve the referenced entity against the observed scene.
Action inference	“The drawer should be closed.”	Action A_k	Infer the action from the specified entity and desired state.
Task decomposition	“Put all the fruit in the bowl.”	Event structure E_τ	Expand the compact objective into the required grounded events.
Intent inference	“I need room on this table.”	Physical goal and event structure	Infer the intended physical goal, relevant entities, and grounded event sequence.

TABLE VIII: Reasoning operations required to recover executable structure left unresolved by a specification. These operations may co-occur within a single specification.

Required reasoning	Success	Progress	Success gap	N
Visual grounding	36.8%	0.51	4.6%	72
Action inference	35.1%	0.49	6.2%	78
Task decomposition	20.4%	0.36	17.8%	64
Intent inference	11.6%	0.27	25.3%	37

TABLE IX: VLA performance by the reasoning required to recover the grounded task structure. Success gap is measured against the matched fully grounded command.

scene, while an omitted action can often be inferred from an explicitly specified object and desired state.

c) Structural Reconstruction Is Harder.: Task decomposition incurs a substantially larger success gap of 17.8%, while intent inference produces the largest gap at 25.3%. These operations require reconstructing more than a locally unresolved argument. Decomposition must recover the latent event sequence underlying a compressed objective, while intent inference must first determine the physical goal itself before the corresponding grounded events can be constructed. The same ordering appears in partial progress.

d) Implication.: The difficulty of an information gap therefore depends not only on how much structure is unresolved, but on what must be inferred from the remaining context. Local ambiguities can often be resolved when language and observation strongly constrain the missing value, whereas reconstructing a physical goal or multi-event execution structure is substantially less reliable. This reasoning-based view complements the main structural analysis by explaining why information gaps of similar apparent size can have very different behavioral costs.